# Encoder models and tasks

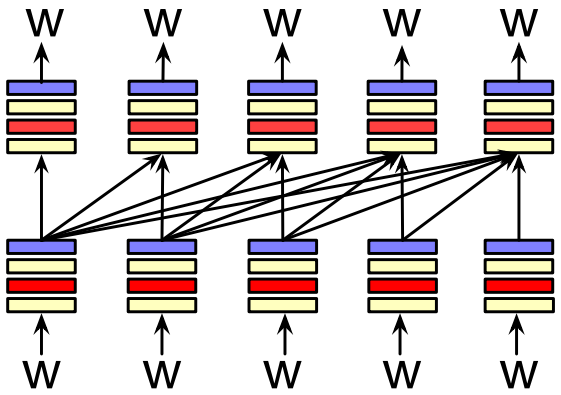
## NLP Week 9

**Thanks to Dan Jurafsky for most of the slides this week!**

# Plan for today

1. Encoder models
2. Masked language modeling
3. Classification with encoders
4. Information Retrieval with encoders
5. *Group exercises*

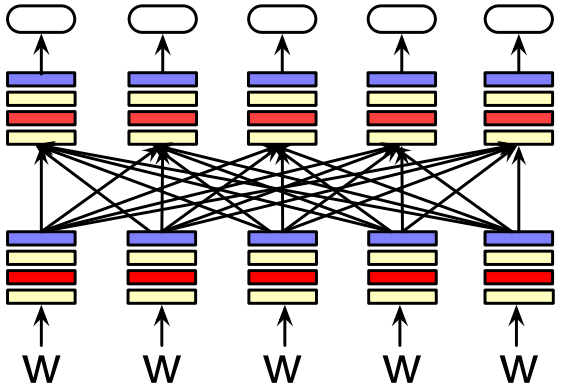
# Three architectures for large language models




**Decoders**

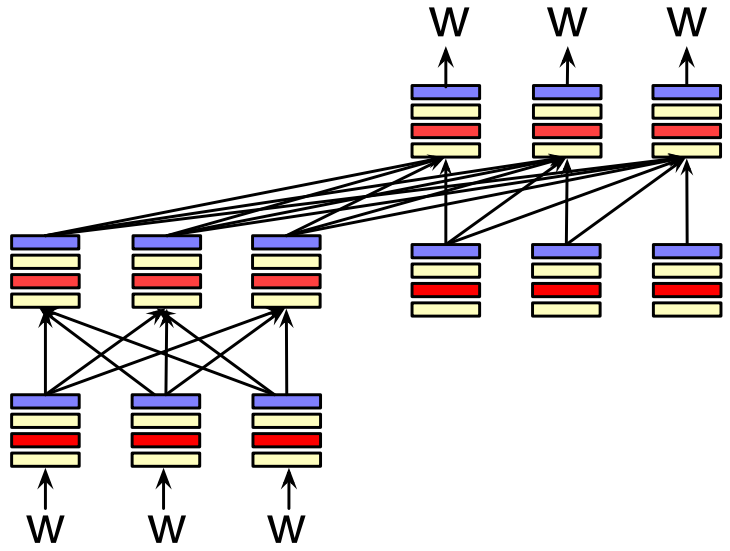
GPT, Claude, Llama

Mixtral




**Encoders**

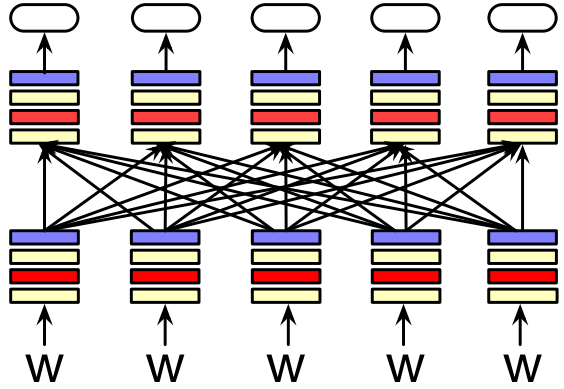
BERT family, HuBERT




**Encoder-decoders**

Flan-T5, Whisper

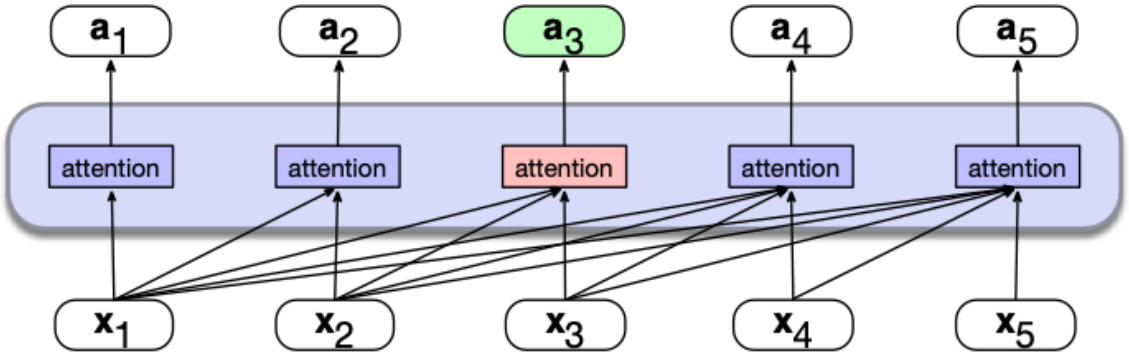
# Encoders



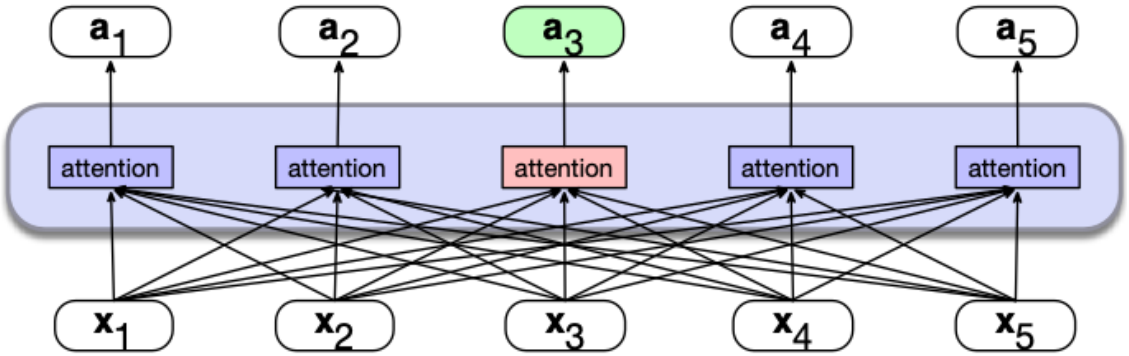

- Masked Language Models (MLMs)
- BERT family

- Trained by predicting words from surrounding words on both sides
- Are usually **finetuned** (trained on supervised data) for classification tasks.

# Bidirectional Self-Attention




a) A causal self-attention layer




b) A bidirectional self-attention layer

# We just remove the mask

Casual self-attention

N

| | | | |
|---|---|---|---|
| q1·k1 | −∞ | −∞ | −∞ |
| q2·k1 | q2·k2 | −∞ | −∞ |
| q3·k1 | q3·k2 | q3·k3 | −∞ |
| q4·k1 | q4·k2 | q4·k3 | q4·k4 |

N

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)\right)$$

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)$$

# We just remove the mask

Casual self-attention

N

| | | | |
|---|---|---|---|
| q1·k1 | −∞ | −∞ | −∞ |
| q2·k1 | q2·k2 | −∞ | −∞ |
| q3·k1 | q3·k2 | q3·k3 | −∞ |
| q4·k1 | q4·k2 | q4·k3 | q4·k4 |

N

$$\mathbf{A} = \text{softmax}\left(\mathbb{M}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)\right)$$

Bidirectional self-attention

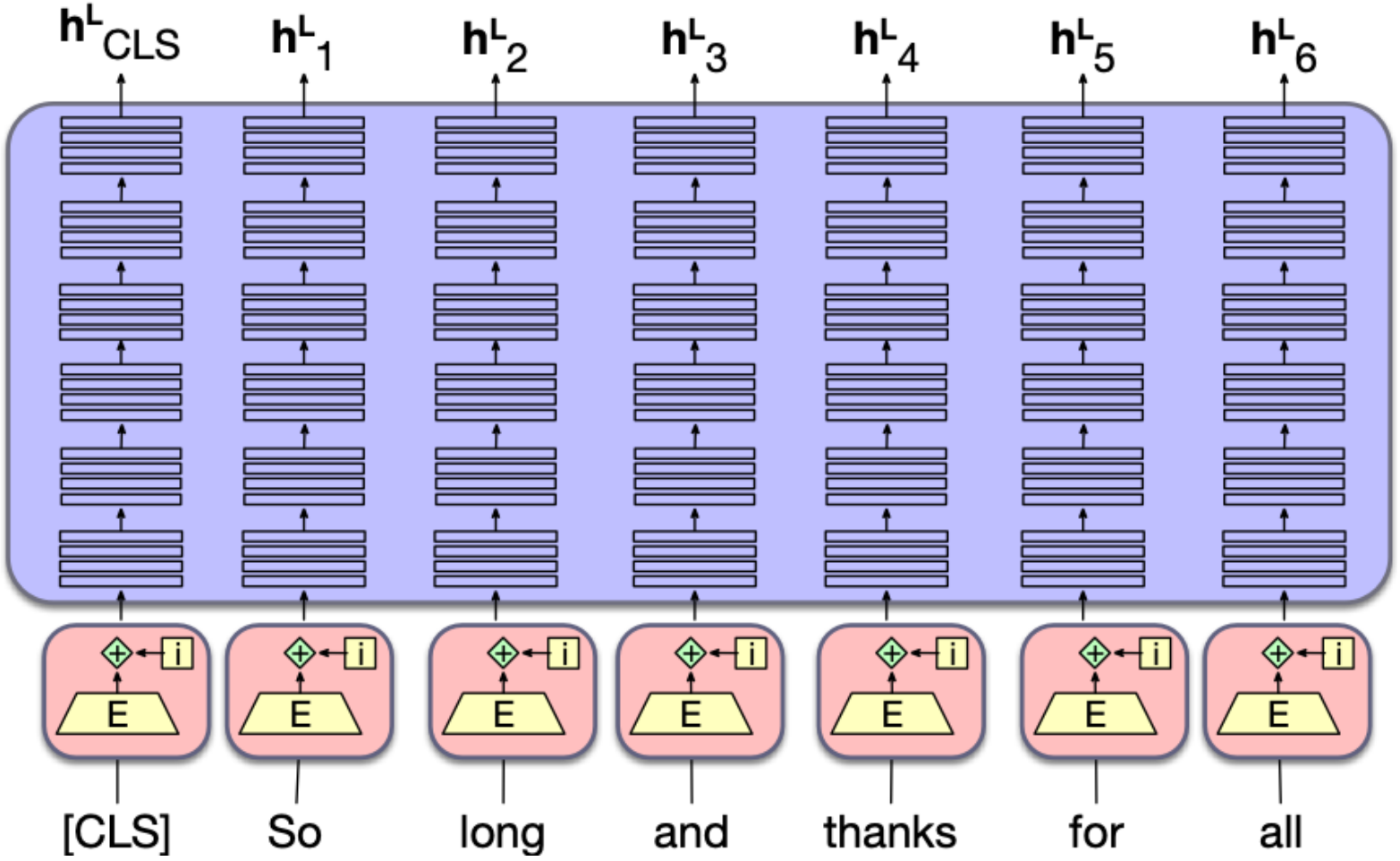
N

| | | | |
|---|---|---|---|
| q1·k1 | q1·k2 | q1·k3 | q1·k4 |
| q2·k1 | q2·k2 | q2·k3 | q2·k4 |
| q3·k1 | q3·k2 | q3·k3 | q3·k4 |
| q4·k1 | q4·k2 | q4·k3 | q4·k4 |

N

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d^k}}\right)$$

# BERT




# Bidirectional Encoder Representations from Transformers

**BERT (Devlin et al., 2019)**

- 30,000 English-only tokens (WordPiece tokenizer)
- Input context window $N$=512 tokens, and model dimensionality $d$=768
- $L$=12 layers of transformer blocks, each with $A$=12 (bidirectional) multihead-attention layers.
- The resulting model has about 100M parameters.

**XLM-RoBERTa (Conneau et al., 2020)**

- 250,000  multilingual tokens (SentencePiece Unigram LM tokenizer)
- Input context window $N$=512 tokens,model dimensionality $d$=1024
- $L$=24 layers of transformer blocks, with $A$=16 multihead attention layers each
- The resulting model has about 550M parameters.

# Masked Language Modeling

- We've seen autoregressive (causal, left-to-right) LMs.
- But what about tasks for which we want to peak at future tokens?
  - Especially true for tasks where we map each input token to an output token
- **Bidirectional encoders** use **unmasked self-attention** to
  - map sequences of input embeddings $\mathbf{x}_1, \ldots, \mathbf{x}_n$
  - to sequences of output embeddings of the same length $\mathbf{h}_1, \ldots, \mathbf{h}_n$
  - where the output vectors have been contextualized using information from the entire input sequence.

# Masked training intuition

- For **left-to-right (causal; decoder-only) LMs**, the model tries to predict the last word from prior words:

  The water of Walden Pond is so beautifully _____

  - And we train it to improve its predictions.
- For **bidirectional masked LMs**, the model tries to predict one or more missing words from all the rest of the words:

  The _____ of Walden Pond _____ so beautifully blue

  - The model generates a probability distribution over the vocabulary for each missing token
  - We use the cross-entropy loss from each of the model's predictions to drive the learning process.

# MLM training in BERT

15% of the tokens are randomly chosen to be part of the masking

Example: "Lunch was **delicious**", if delicious was randomly chosen:

Three possibilities:

1. 80%: Token is replaced with special token [MASK]

   Lunch was **delicious** -> Lunch was **[MASK]**

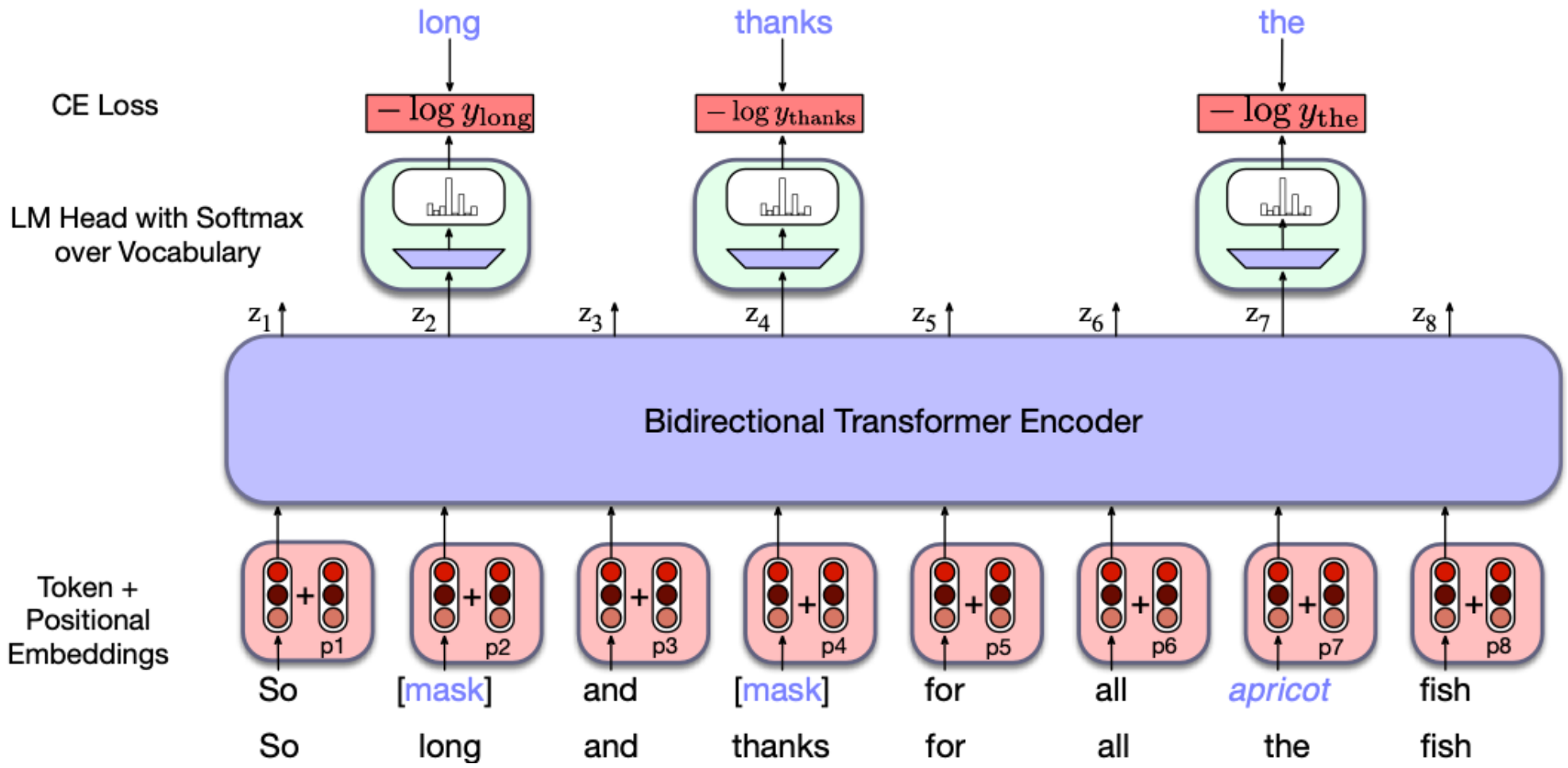
2. 10%: Token is replaced with a random token (sampled from unigram prob)

   Lunch was **delicious** -> Lunch was **gasp**

3. 10%: Token is unchanged

   Lunch was **delicious** -> Lunch was **delicious**

# In detail


long
thanks
the
CE Loss
$-\log y_{\text{long}}$
$-\log y_{\text{thanks}}$
$-\log y_{\text{the}}$
LM Head with Softmax over Vocabulary
$z_1$ $z_2$ $z_3$ $z_4$ $z_5$ $z_6$ $z_7$ $z_8$
Bidirectional Transformer Encoder
Token + Positional Embeddings
p1 p2 p3 p4 p5 p6 p7 p8
So [mask] and [mask] for all apricot fish
So long and thanks for all the fish

# MLM loss

The LM head takes output of final transformer layer L, multiplies it by unembedding layer and turns into probabilities:

$$\mathbf{u}_i = E\mathbf{h}_i^L \qquad \mathbf{y}_i = \text{softmax}(\mathbf{u}_i)$$

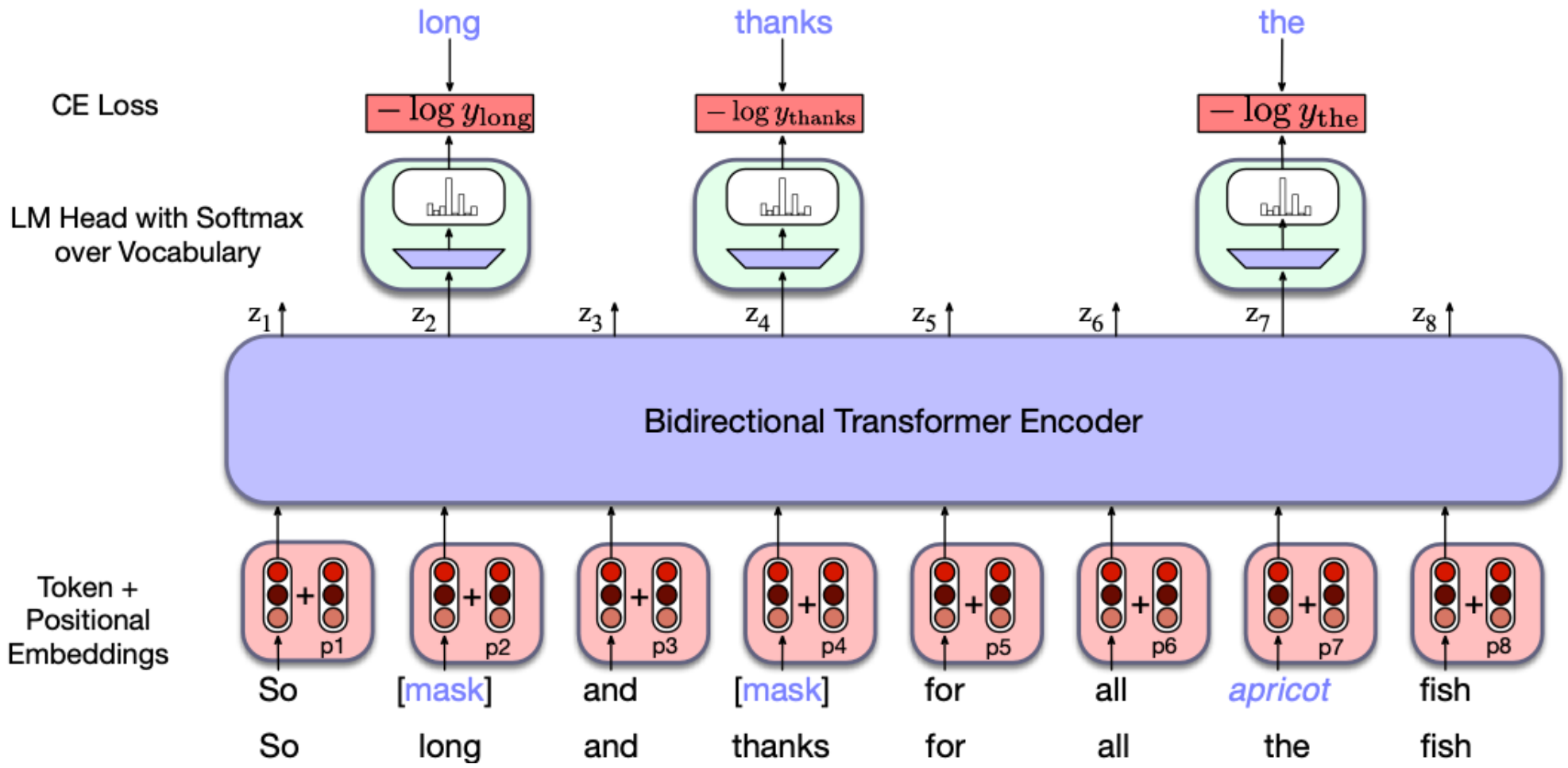
E.g., for the $x_i$ corresponding to "long", the loss is the probability of the correct word *long*, given output $h^L_i$ ):

$$\mathscr{L}_{\text{MLM}}(x_i) = -\log P(x_i \mid \mathbf{h}_i^L)$$

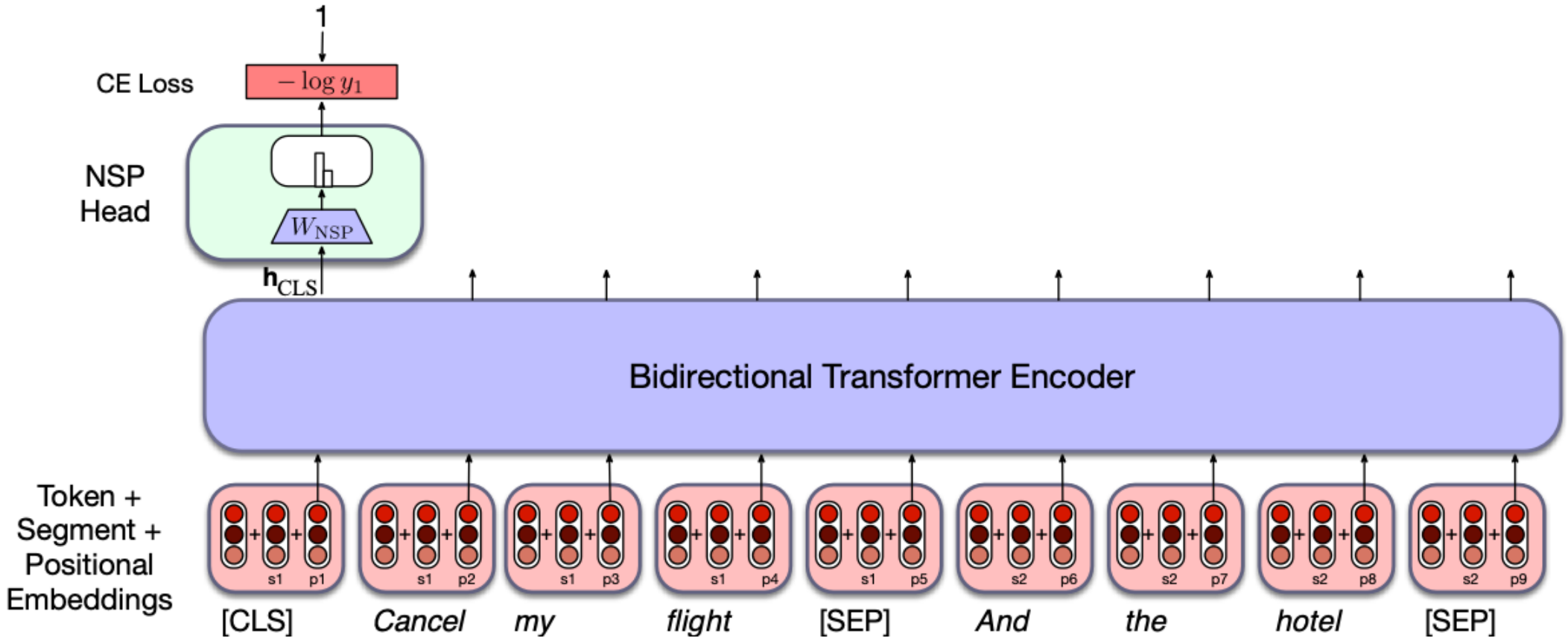
We get the gradients by taking the average of this loss over the batch

$$\mathscr{L}_{\text{MLM}}(x_i) = -\frac{1}{|\mathscr{B}|}\sum_{i\in\mathscr{B}} \log P(x_i \mid \mathbf{h}_i^L)$$

# In detail


long
thanks
the
CE Loss
$-\log y_{\text{long}}$
$-\log y_{\text{thanks}}$
$-\log y_{\text{the}}$
LM Head with Softmax over Vocabulary
$z_1$ $z_2$ $z_3$ $z_4$ $z_5$ $z_6$ $z_7$ $z_8$
Bidirectional Transformer Encoder
Token + Positional Embeddings
p1 p2 p3 p4 p5 p6 p7 p8
So [mask] and [mask] for all apricot fish
So long and thanks for all the fish

# NSP Loss with classification head



# Next Sentence Prediction Loss

Given 2 sentences the model predicts if they are a real pair of adjacent sentences from the training corpus or a pair of unrelated sentences.

BERT introduces two special tokens

- [CLS] is prepended to the input sentence pair,
- [SEP] is placed between the sentences, and also after second sentence

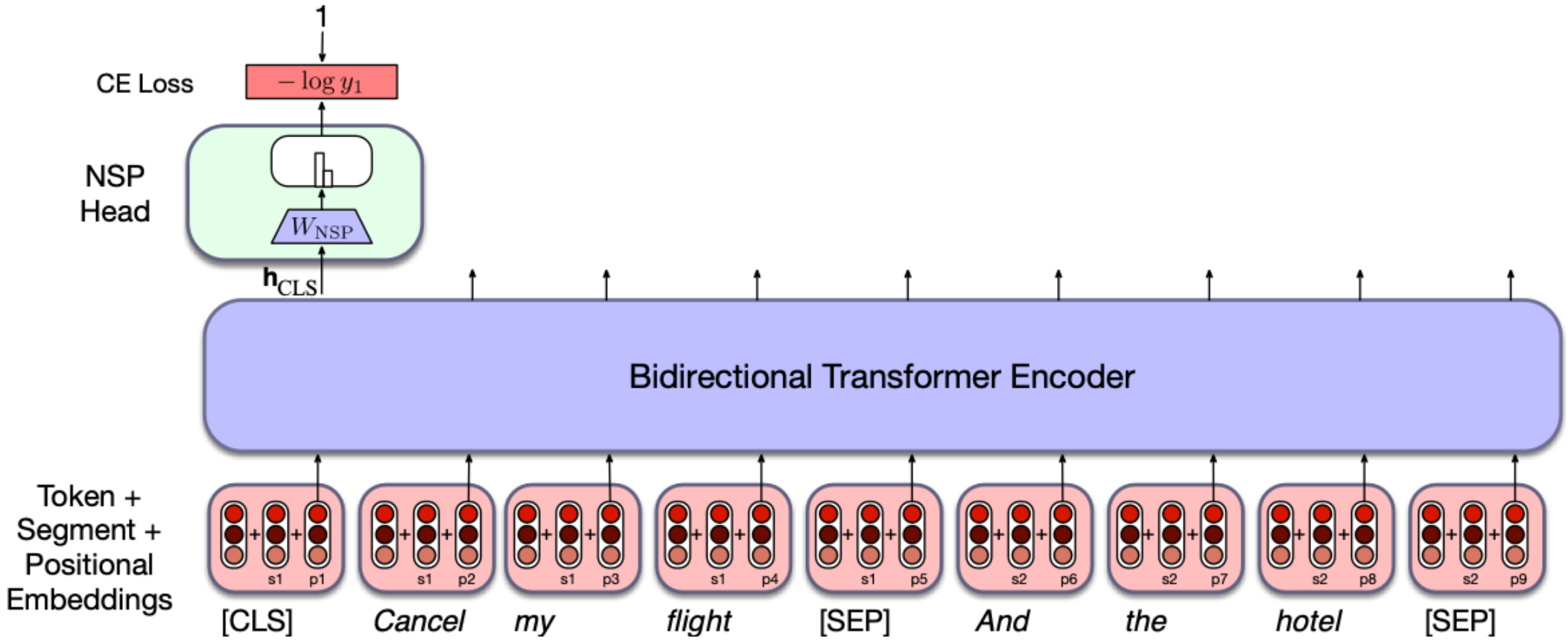
And two more special tokens

- [1st segment] and [2nd segment]
- These are added to the input embedding and positional embedding

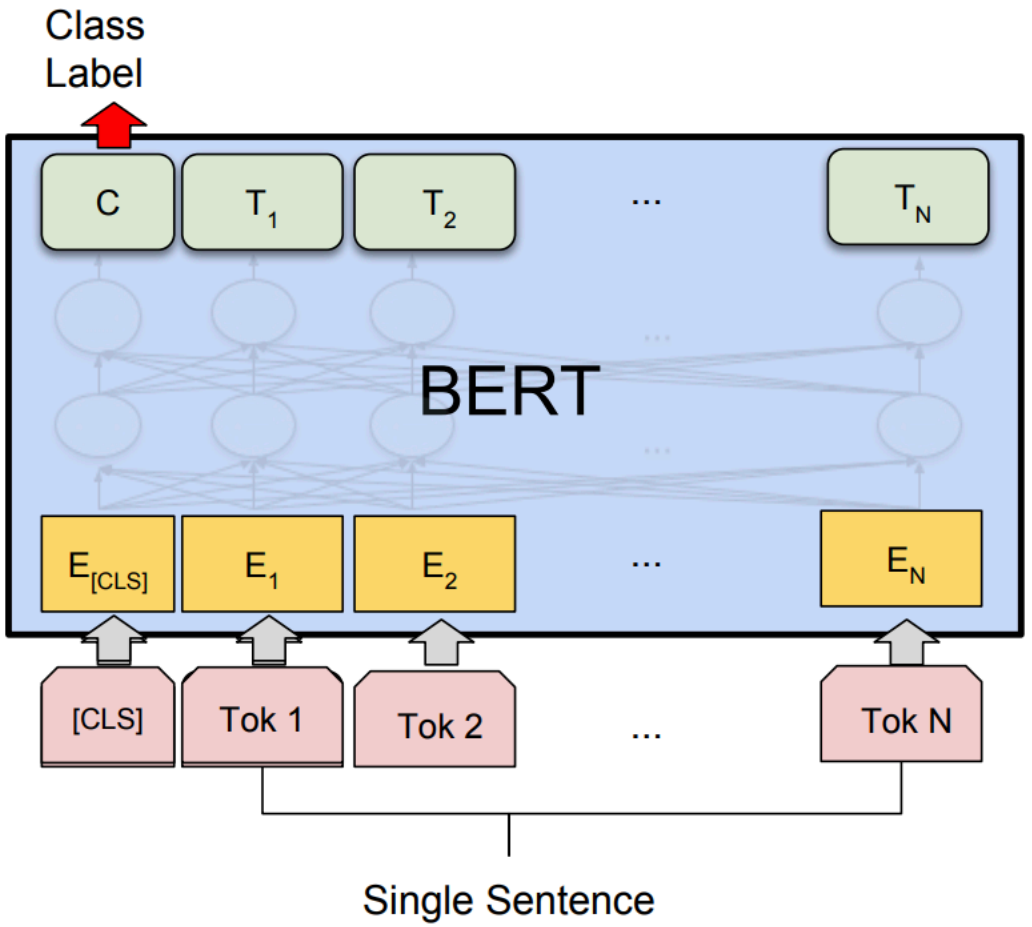
$h^L_{CLS}$ from the final layer [CLS] token is input to classifier head (weights $W_{NSP}$ ) that predicts two classes:.

$$\mathbf{y}_i = \text{softmax}(\mathbf{h}^L_{\text{CLS}})\mathbf{W}_{\text{NSP}}$$

# NSP Loss with classification head



# Using BERT for classification



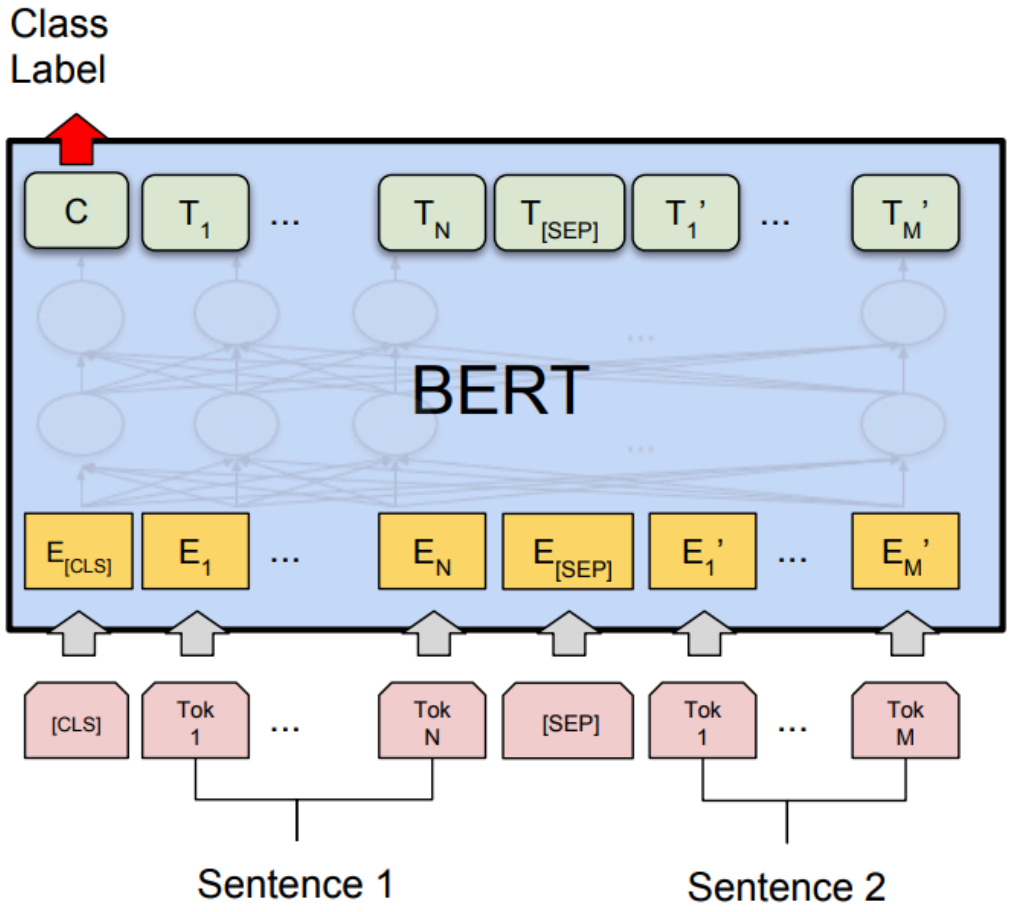

Devlin et al. 2018 - BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

# Sentence classification

Assign a label to a single sentences:

- **Sentiment analysis** (does a given sentence have positive or negative sentiment?)
- **Acceptability judgements** (is a given sentence grammatical?)
- **Hate speech detection** (e.g, does a given sentence express hate or encourages violence?)

# Using BERT for sentence pair classification




Devlin et al. 2018 - BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

# Sequence-Pair classification

Assign a label to pairs of sentences:

- **Paraphrase detection** (are the two sentences paraphrases of each other?)
- **Logical entailment** (does sentence A logically entail sentence B?)
- **Discourse coherence** (how coherent is sentence B as a follow-on to sentence A?)

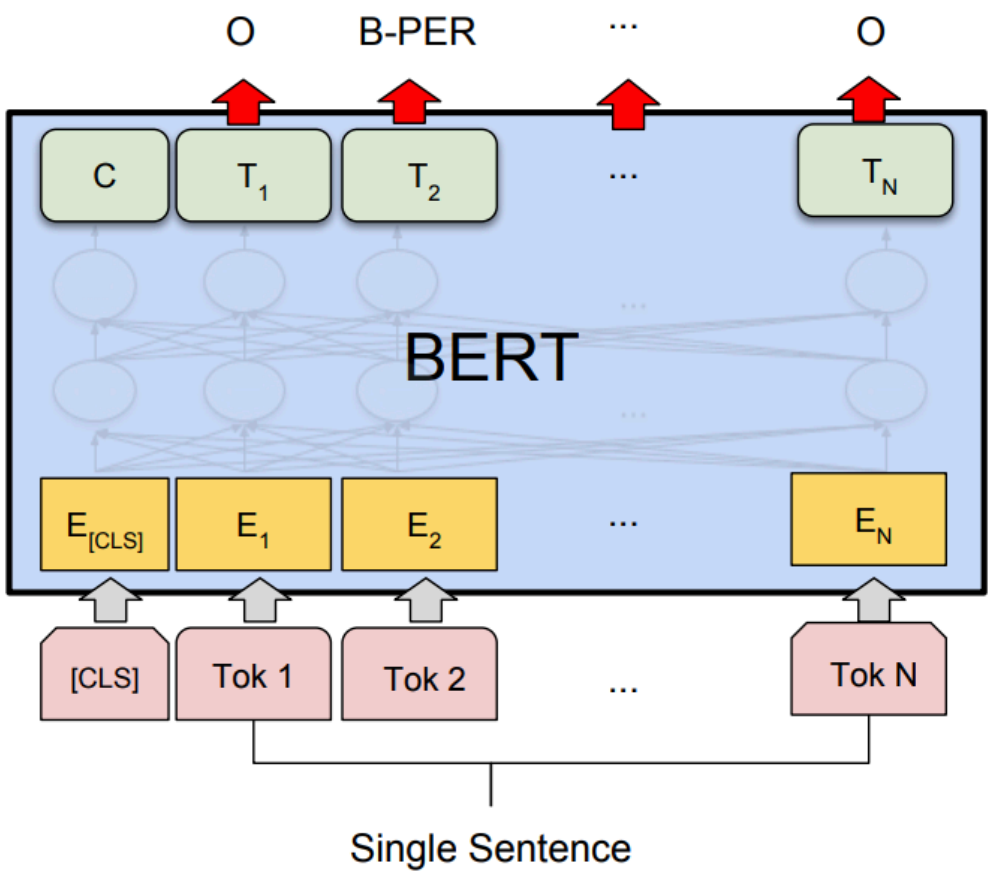
# Example: Natural Language Inference

Pairs of sentences are given one of 3 labels

- `Neutral`
  - a: Jon walked back to the town to the smithy.
  - b: Jon traveled back to his hometown.
- `Contradicts`
  - a: Tourist Information offices can be very helpful.
  - b: Tourist Information offices are never of any help.
- `Entails`
  - a: I'm confused.
  - b: Not all of it is very clear to me.

How? —> pass the premise/hypothesis pairs through BERT and use the output vector for the [CLS] token as the input to the classification head .

# Using BERT for sequence labeling




Devlin et al. 2018 - BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

# Using BERT for sequence labeling

Assign a label from a small fixed set of labels to each token in the sequence.

- Named entity recognition
- Part of speech tagging

# Named Entity Recognition

A **named entity** is anything that can be referred to with a proper name: a person, a location, an organization

**Named entity recognition (NER):** find spans of text that constitute proper names and tag the type of the entity

| Type | Tag | Sample Categories | Example sentences |
|---|---|---|---|
| People | PER | people, characters | **Turing** is a giant of computer science. |
| Organization | ORG | companies, sports teams | The **IPCC** warned about the cyclone. |
| Location | LOC | regions, mountains, seas | **Mt. Sanitas** is in **Sunshine Canyon**. |
| Geo-Political Entity | GPE | countries, states | **Palo Alto** is raising the fees for parking. |

# Named Entity Recognition

Citing high fuel prices, [ORG **United Airlines**] said [TIME **Friday**] it has increased fares by [MONEY **$6**] per round trip on flights to some cities also served by lower-cost carriers. [ORG **American Airlines**], a unit of [ORG **AMR Corp.**], immediately matched the move, spokesman [PER **Tim Wagner**] said. [ORG **United**], a unit of [ORG **UAL Corp.**], said the increase took effect [TIME **Thursday**] and applies to most routes where it competes against discount carriers, such as [LOC **Chicago**] to [LOC **Dallas**] and [LOC **Denver**] to [LOC **San Francisco**].

# BIO Tagging

Ramshaw and Marcus (1995)

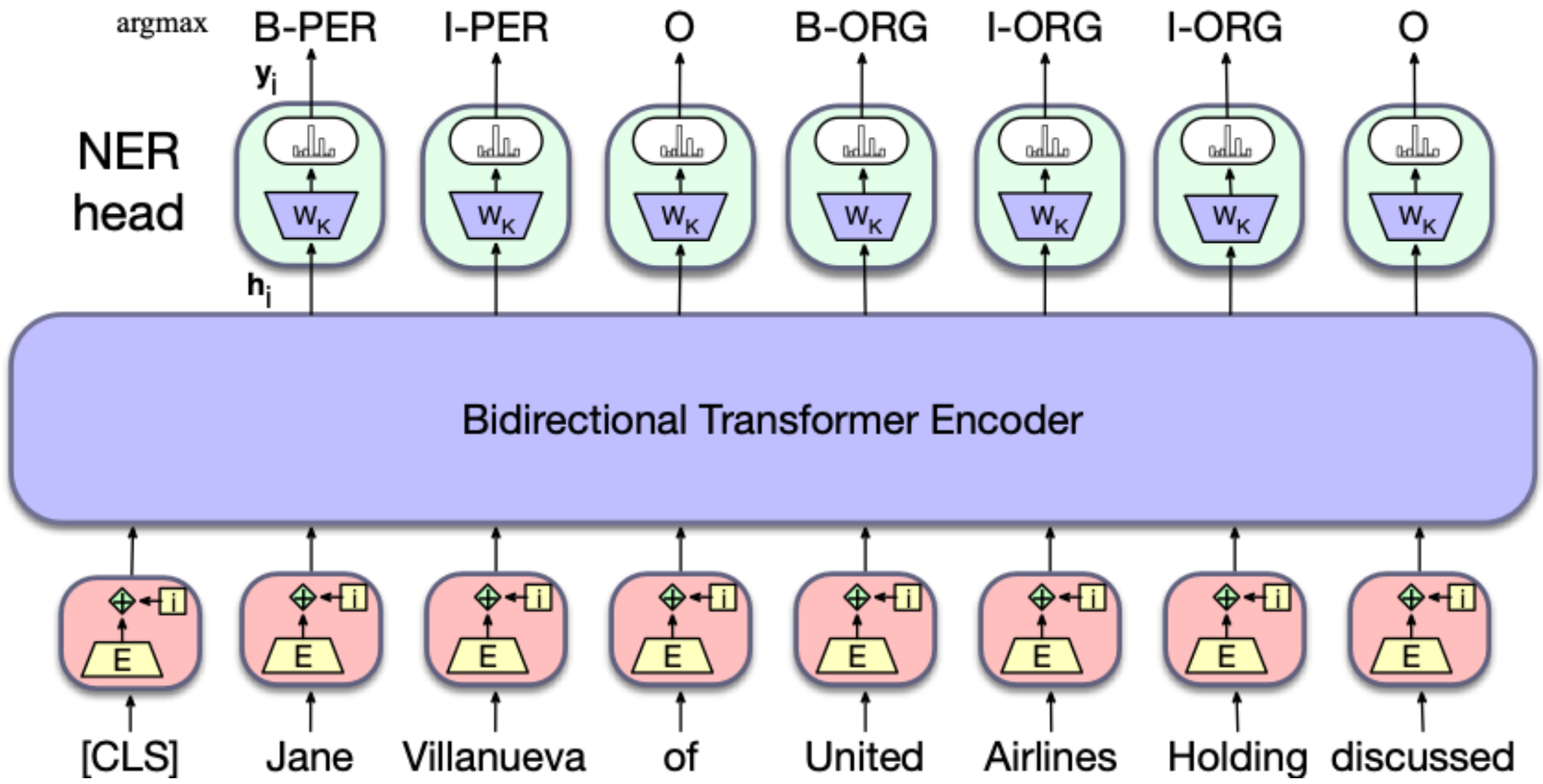
A method that lets us turn a segmentation task (finding boundaries of entities) into a classification task

[PER **Jane Villanueva** ] of [ORG **United**] , a unit of [ORG **United Airlines Holding**] , said the fare applies to the [LOC **Chicago** ] route.

| Words | IO Label | BIO Label | BIOES Label |
|---|---|---|---|
| Jane | I-PER | B-PER | B-PER |
| Villanueva | I-PER | I-PER | E-PER |
| of | O | O | O |
| United | I-ORG | B-ORG | B-ORG |
| Airlines | I-ORG | I-ORG | I-ORG |
| Holding | I-ORG | I-ORG | E-ORG |
| discussed | O | O | O |
| the | O | O | O |
| Chicago | I-LOC | B-LOC | S-LOC |
| route | O | O | O |
| . | O | O | O |

# Sequence labeling

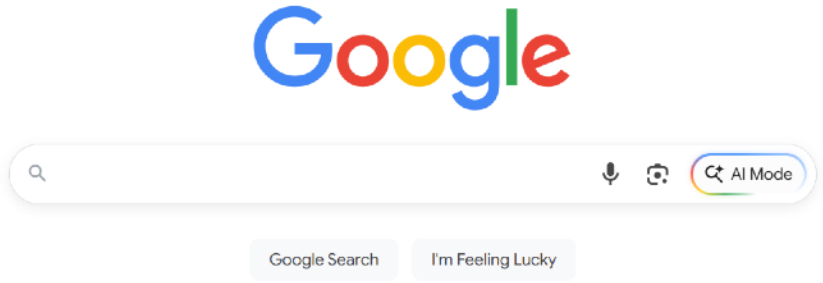
$$\mathbf{y_i} = \text{softmax}(\mathbf{h_i^L W_K})$$
$$\mathbf{t_i} = \text{argmax}_k(\mathbf{y}_i)$$



# Information retrieval

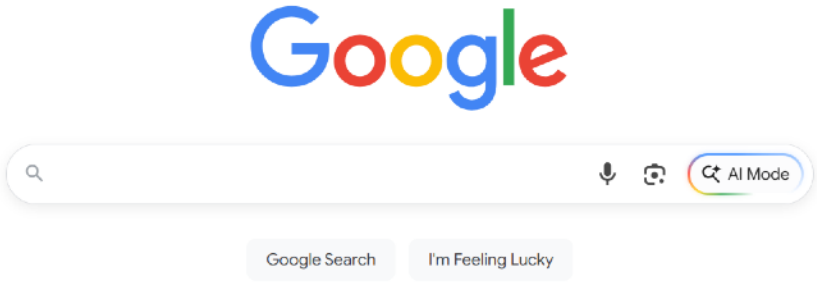
A user has an information need and has some collection of documents. They wants to find a **relevant** document (or documents) in the collection to satisfy their need.

# Information retrieval at large



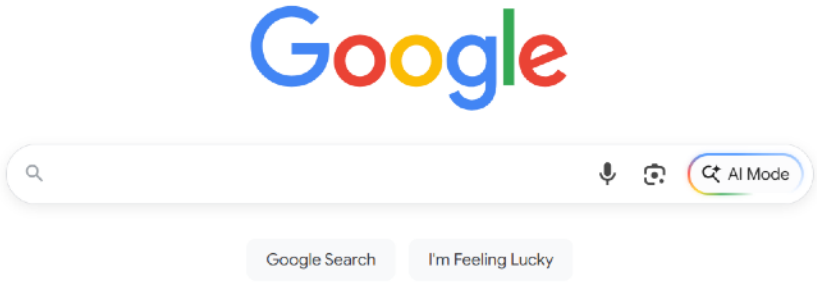
# Information retrieval at large

- Searching the web



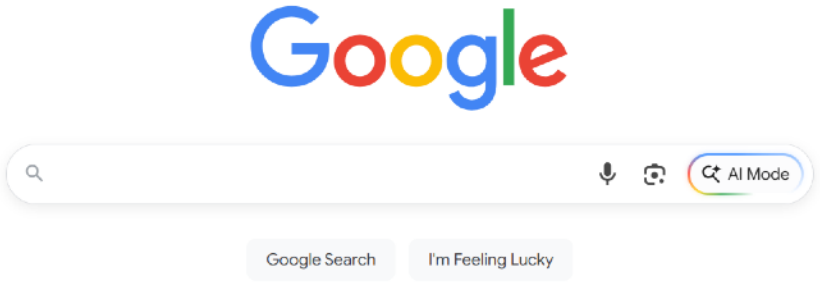
# Information retrieval at large

- Searching the web
- Searching our email



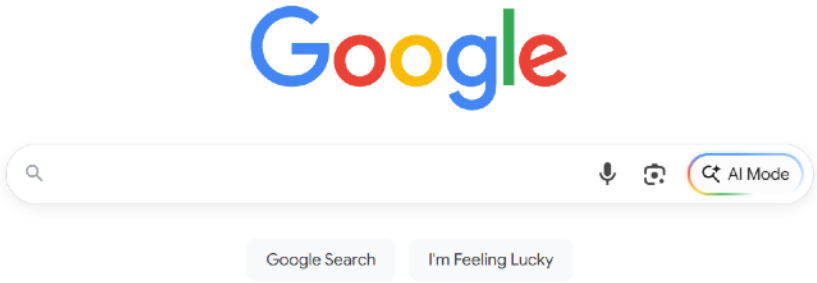
# Information retrieval at large

- Searching the web
- Searching our email
- Searching corporate documents



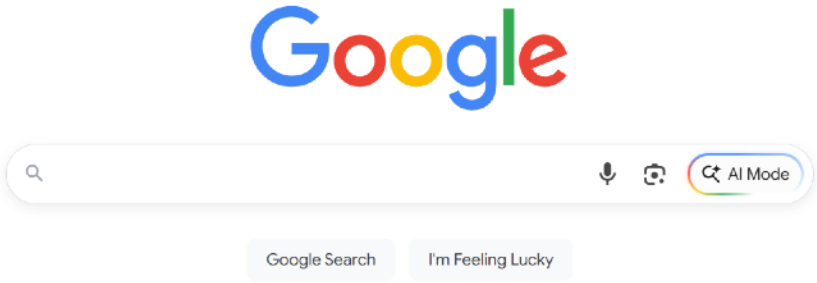
# Information retrieval at large

- Searching the web
- Searching our email
- Searching corporate documents
- Searching personal medical records



# Information retrieval at large

- Searching the web
- Searching our email
- Searching corporate documents
- Searching personal medical records
- And also, part of LLMs in retrieval-augmented generation (RAG)



# In most cases we do "ranked retrieval"

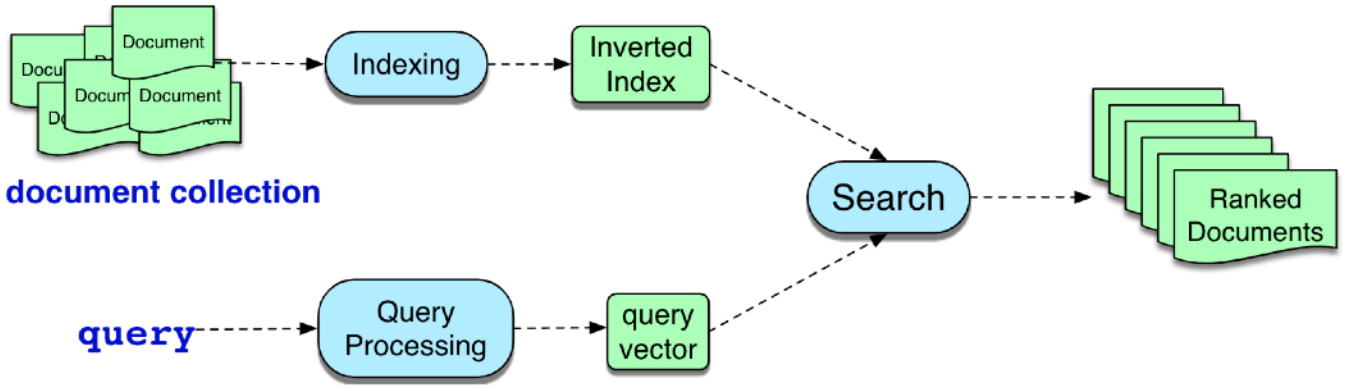
The retriever returns top-k documents.

These are **ranked using a document relevance score.**

Goal is to assign a score to each document for whether it meets the user's information need. We just approximate this by the textual similarity between the query and the document.

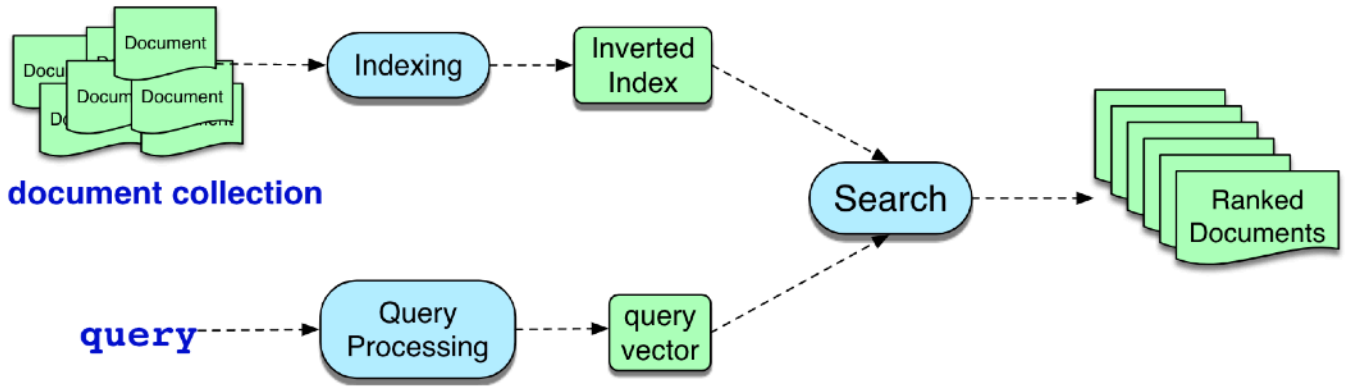
# Information retrieval

- Retrieve *media* based on a user's *information need*
- We consider a special case of IR: **ad-hoc retrieval**
- A user provides a question to a system, which then returns one or more documents from a collection of documents



# Information retrieval

- **Document**: the unit of text that the IR system indexes
- **Collection**: set of documents
- **Term**: a word (or paragraph) that occurs in the collection
- **Query**: information need expressed as a set of terms



# Two architectures

**Sparse retrieval**

- represent query and doc as vectors of word counts
- weighted by tf-idf, BM25

**Dense retrieval**

- Use LLM to represent query and doc as embeddings

In both cases, similarity is dot product or cosine between query and document representations

Recall from Week 4

# Embedding types

- **There are three main types of embeddings used in NLP and they are created using three different methods:**

  1. Sparse embeddings and term-document frequency

  2. Dense embeddings and Word2Vec algorithms

  3. Contextual word embeddings and language modeling

Recall from Week 4

# Sparse embeddings and tf-idf scoring

- Weighted value for each term

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_f$$

- How do we score queries and documents? Cosine similarity

$$\text{score}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}||\mathbf{d}|}$$

Recall from Week 4

# Term frequency (tf) in the tf-idf algorithm

We could imagine using raw count:

$$\text{tf}_{t,d} = \text{count}(t,d)$$

But instead of using raw count, we usually squash a bit:

$$\text{tf}_{t,d} = \begin{cases} 1 + \log_{10} \text{count}(t,d) & \text{if } \text{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Recall from Week 4

# Inverse document frequency (idf)

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

$\text{df}_t$ *is* the number of documents *t* occurs in.

(note this is not collection frequency: total count across all documents)

N is the total number of documents in the collection

# tf-idf scoring

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|}$$

- Rewrite the dot-product as a sum of products

$$\text{score}(q,d) = \sum_{t\in\mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i\in q}\text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i\in d}\text{tf-idf}^2(d_i,d)}}$$

# tf-idf scoring

$$\text{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

| | |
|---|---|
| **Query**: | sweet love |
| **Doc 1**: | Sweet sweet nurse! Love? |
| **Doc 2**: | Sweet sorrow |
| **Doc 3**: | How sweet is love? |
| **Doc 4**: | Nurse! |

| | Query | | | | | |
|---|---|---|---|---|---|---|
| **word** | **cnt** | **tf** | **df** | **idf** | **tf-idf** | **n'lized** = tf-idf/$\lvert q \rvert$ |
| sweet | 1 | 1 | 3 | 0.125 | 0.125 | 0.383 |
| nurse | 0 | 0 | 2 | 0.301 | 0 | 0 |
| love | 1 | 1 | 2 | 0.301 | 0.301 | 0.924 |
| how | 0 | 0 | 1 | 0.602 | 0 | 0 |
| sorrow | 0 | 0 | 1 | 0.602 | 0 | 0 |
| is | 0 | 0 | 1 | 0.602 | 0 | 0 |
| $\lvert q \rvert = \sqrt{.125^2 + .301^2} = .326$ | | | | | | |

# tf-idf scoring

$$\text{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

| | |
|---|---|
| **Query**: | sweet love |
| **Doc 1**: | Sweet sweet nurse! Love? |
| **Doc 2**: | Sweet sorrow |
| **Doc 3**: | How sweet is love? |
| **Doc 4**: | Nurse! |

| | Document 1 | | | | | Document 2 | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **word** | **cnt** | **tf** | **tf-idf** | **n'lized** | **× q** | **cnt** | **tf** | **tf-idf** | **n'lized** | **×q** |
| sweet | 2 | 1.301 | 0.163 | 0.357 | **0.137** | 1 | 1.000 | 0.125 | 0.203 | **0.0779** |
| nurse | 1 | 1.000 | 0.301 | 0.661 | 0 | 0 | 0 | 0 | 0 | 0 |
| love | 1 | 1.000 | 0.301 | 0.661 | **0.610** | 0 | 0 | 0 | 0 | **0** |
| how | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| sorrow | 0 | 0 | 0 | 0 | 0 | 1 | 1.000 | 0.602 | 0.979 | 0 |
| is | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| $\lvert d_1 \rvert = \sqrt{.163^2 + .301^2 + .301^2} = .456$ | | | | | | $\lvert d_2 \rvert = \sqrt{.125^2 + .602^2} = .615$ | | | | |
| Cosine: $\sum$ of column: | | | | | **0.747** | Cosine: $\sum$ of column: | | | | **0.0779** |

# BM25

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

- Introduce parameters k and b

$$\text{score}(q,d) = \sum_{t \in q} \overbrace{\log\left(\frac{N}{df_t}\right)}^{\text{IDF}} \overbrace{\frac{tf_{t,d}}{k\left(1-b+b\left(\frac{|d|}{|d_{\text{avg}}|}\right)\right)+tf_{t,d}}}^{\text{weighted tf}}$$

**k** (Term Frequency Saturation): Controls how much additional occurrences of a term increase the score

**b** (Document Length Normalization): Controls how much the length of a document affects the score, preventing long documents from being favored simply because they contain more word

# Efficient sparse retrieval

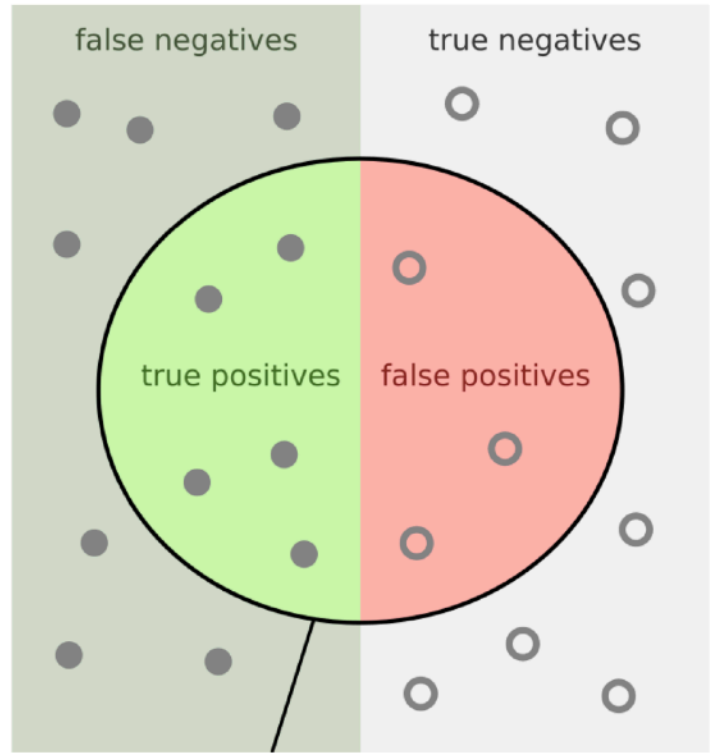
- Efficiently find documents that contain terms of interest, by ignoring documents that don't contain any of the query words!
- **Inverted index**
  - Dictionary: list of terms (+ document frequency)
  - Postings list: list of document ids that contain the term (+ term frequency in each of the documents)

| | |
|---|---|
| **Query**: | sweet love |
| **Doc 1**: | Sweet sweet nurse! Love? |
| **Doc 2**: | Sweet sorrow |
| **Doc 3**: | How sweet is love? |
| **Doc 4**: | Nurse! |

| | | |
|---|---|---|
| how {1} | → | 3 [1] |
| is {1} | → | 3 [1] |
| love {2} | → | 1 [1] → 3 [1] |
| nurse {2} | → | 1 [1] → 4 [1] |
| sorry {1} | → | 2 [1] |
| sweet {3} | → | 1 [2] → 2 [1] → 3 [1] |

# Precision and Recall

- Heuristic classifiers tend to have high **precision** but very low **recall.**
- **Classifier accuracy is measured with precision and recall:**




How many relevant items are retrieved?

Recall = 

How many retrieved items are relevant?

Precision = 

Recall = TP / (**FN** +TP)  Precision = TP / (**FP** +TP)

# Precision and recall in IR

- Each document is either relevant or not relevant
- **Precision**: Fraction of retrieved documents that are relevant
- **Recall**: Fraction of all relevant documents that are retrieved
- Let T be the set of returned documents, R are the relevant documents in T, N are the irrelevant documents in T, U are all relevant documents in the collection

$$Precision = \frac{|R|}{|T|} \qquad Recall = \frac{|R|}{|U|}$$

# Evaluation of information retrieval systems

- We care about the rank of the target document(s)
- **Precision@k:** fraction of relevant documents seen at a particular rank k
- **Recall@k:** fraction of relevant documents found at a particular rank k

| Rank | Judgment | $\text{Precision}_{Rank}$ | $\text{Recall}_{Rank}$ |
|---|---|---|---|
| 1 | R | 1.0 | .11 |
| 2 | N | .50 | .11 |
| 3 | R | .66 | .22 |
| 4 | N | .50 | .22 |
| 5 | R | .60 | .33 |
| 6 | R | .66 | .44 |
| 7 | N | .57 | .44 |
| 8 | R | .63 | .55 |
| 9 | N | .55 | .55 |
| 10 | N | .50 | .55 |

# Evaluation of information retrieval systems

- **Mean average precision** (MAP)
  - Iterate over the ranked list top to bottom
  - Note the precision **only** at positions where a relevant item has been encountered
  - Average these precisions over the return set
  - Formally: Let $R_r$ be the set of relevant documents at or above rank $r$
  - **Average precision**:

$$\mathrm{AP} = \frac{1}{|R_r|} \sum_{d \in R_r} \mathrm{Precision}_r(d)$$

    - $\mathrm{Precision}_r(d)$ precision measured at rank where d was found

# Evaluation of information retrieval systems

- **Mean average precision** (MAP)
  - **Average precision**:

$$\mathrm{AP} = \frac{1}{|R_r|} \sum_{d \in R_r} \mathrm{Precision}_r(d)$$

    - $\mathrm{Precision}_r(d)$ precision measured at rank where d was found
- Given a set of queries Q:

$$\mathrm{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP}(q)$$

# Evaluation of information retrieval systems

- **Mean average precision** (MAP)

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

| Rank | Judgment | $\text{Precision}_{Rank}$ | $\text{Recall}_{Rank}$ |
|---|---|---|---|
| 1 | R | 1.0 | .11 |
| 2 | N | .50 | .11 |
| 3 | R | .66 | .22 |
| 4 | N | .50 | .22 |
| 5 | R | .60 | .33 |
| 6 | R | .66 | .44 |
| 7 | N | .57 | .44 |
| 8 | R | .63 | .55 |
| 9 | N | .55 | .55 |
| 10 | N | .50 | .55 |

- Relevant docs at: 1, 3, 5, 6, 8
- Precisions: 1.0, 0.66, 0.60, 0.66, 0.63
- AP = (1.0 + 0.66 + 0.60 + 0.66 + 0.63) / 5 = 3.55 / 5 = 0.71

# Problems with sparse IR

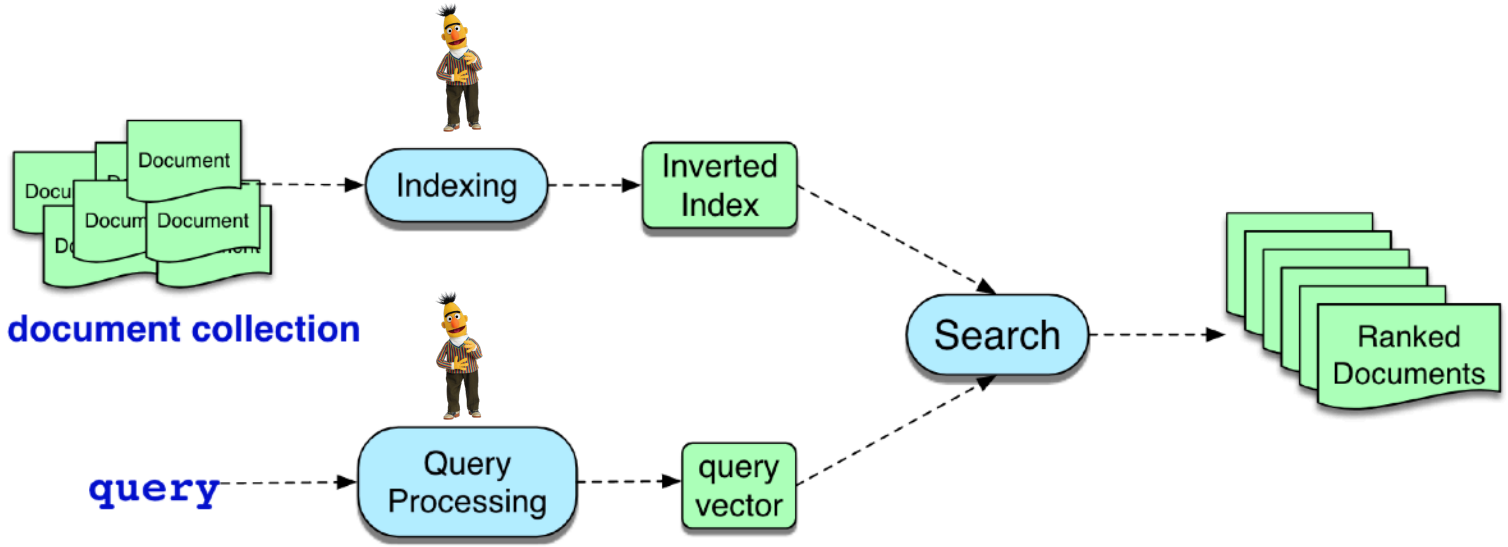
The **vocabulary mismatch problem**

- tf-idf or BM25 cosine similarities only work if there is **exact word overlap** between query and doc!
- But query-writer can't know the exact words the doc might include!

# Dense Retrieval

Instead of representing query and documents with count vectors
Represent both with embeddings!

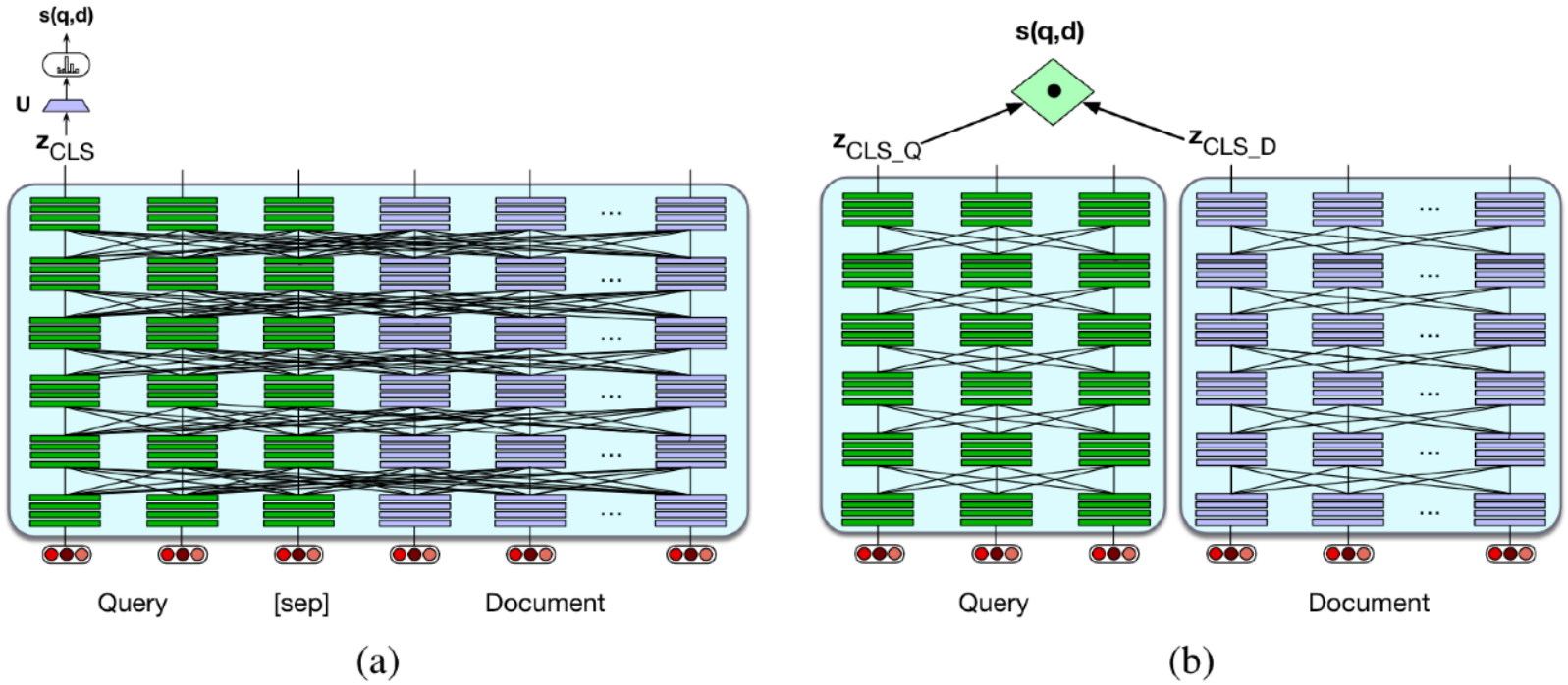
# Information retrieval with pre-trained LMs




- Instead of using word-count vectors, use dense vectors from pre-trained language models, e.g., BERT

# Information retrieval with pre-trained LMs

- Two potential ways to index queries and documents
  - Jointly encode query and each document
  - Bi-encoder: encode query and documents independently

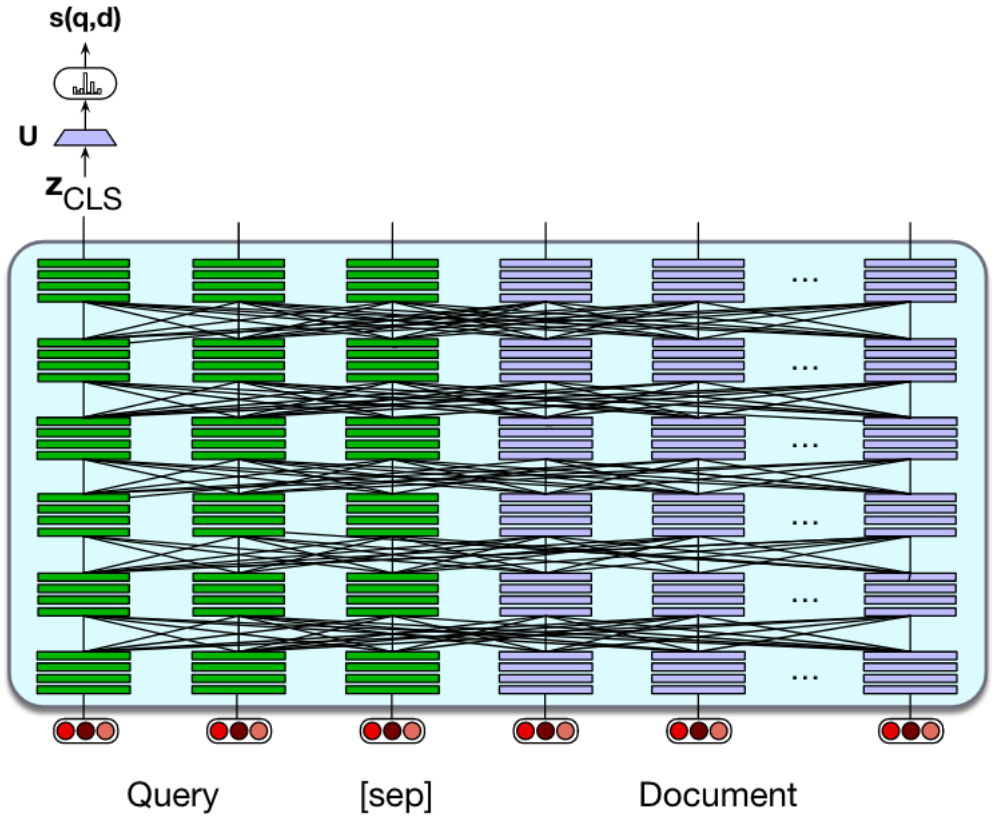


# Information retrieval with pre-trained LMs

- Jointly encode query and every document

$$\mathbf{z} = \text{BERT}(q; [\text{SEP}]; d)[\text{CLS}]$$

$$\text{score}(\mathbf{z}) = \text{softmax}(\mathbf{U}\mathbf{z})$$

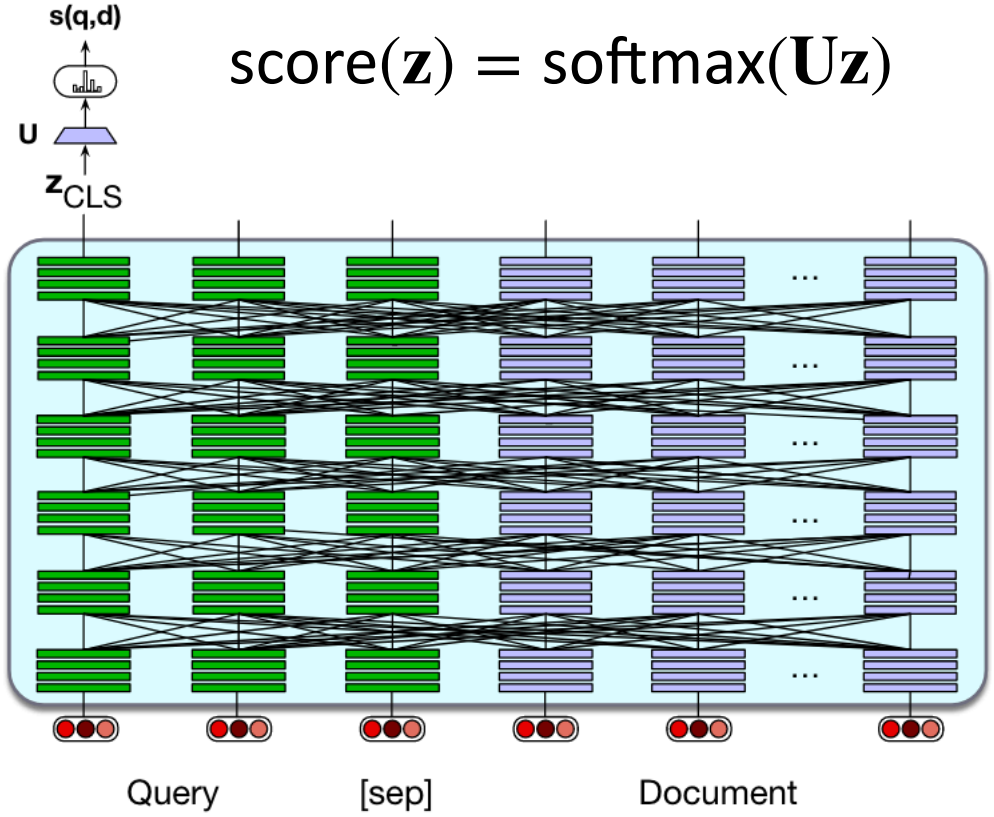


# Information retrieval with pre-trained LMs

- This approach is expensive
- Need to re-encode every document for each query
- This quickly becomes impractical when dealing with large document collections

$$\mathbf{z} = \text{BERT}(q; [\text{SEP}]; d)[\text{CLS}]$$

$$\text{score}(\mathbf{z}) = \text{softmax}(\mathbf{U}\mathbf{z})$$



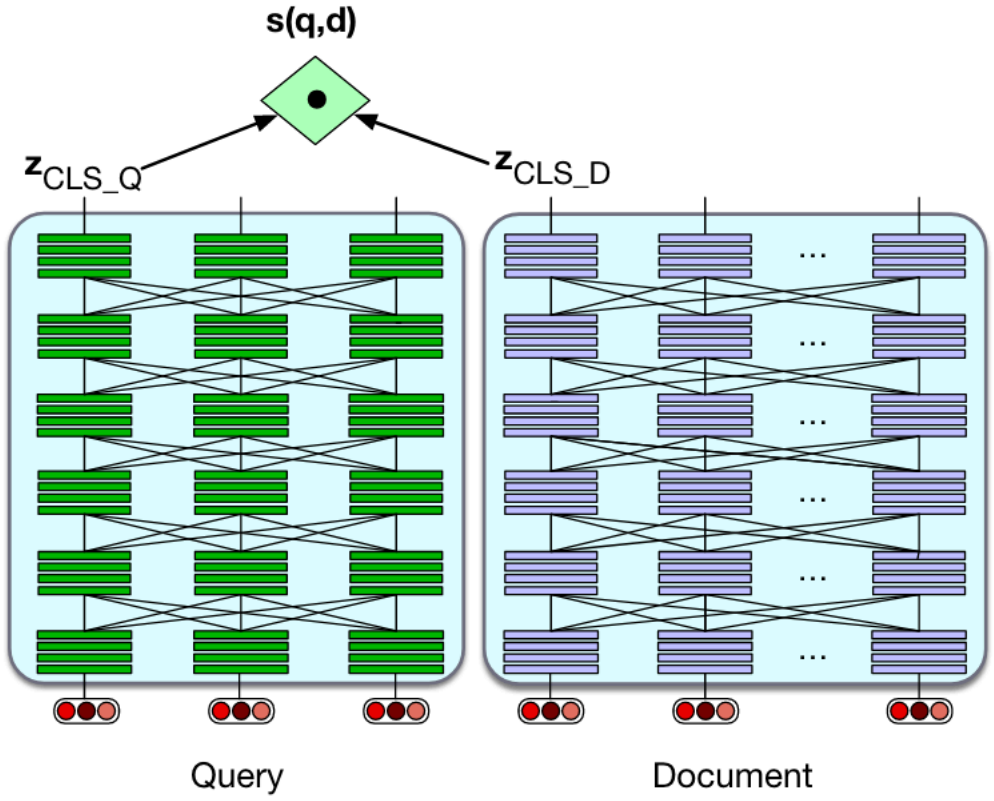
# Information retrieval with pre-trained LMs

- **Bi-encoder**: encode queries and documents independently

$$\mathbf{z}_q = \text{BERT}_{\text{query}}(q)[\text{CLS}]$$

$$\mathbf{z}_d = \text{BERT}_{\text{doc}}(d)[\text{CLS}]$$

$$\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$$



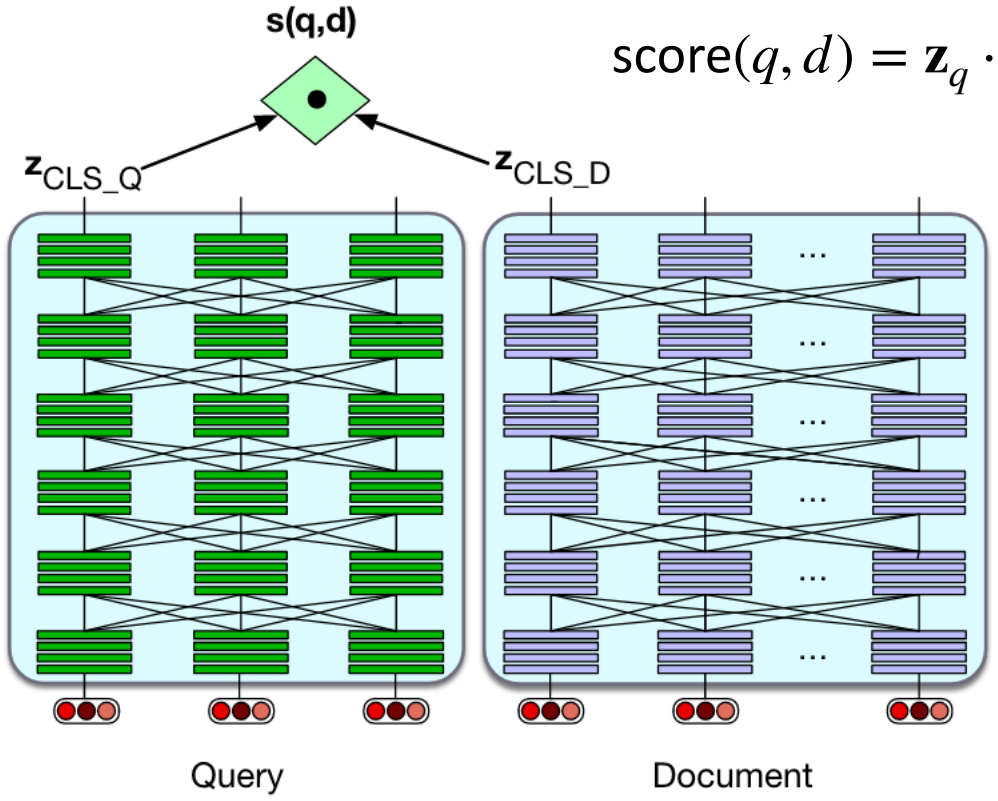
# Information retrieval with pre-trained LMs

- Documents are encoded in advance
- When a new query comes in, we only have to encode that query
- Much more efficient but tends to give worse results
- Question: How to combine the two approaches?

$$\mathbf{z}_q = \text{BERT}_{\text{query}}(q)[\text{CLS}]$$

$$\mathbf{z}_d = \text{BERT}_{\text{doc}}(d)[\text{CLS}]$$

$$\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$$



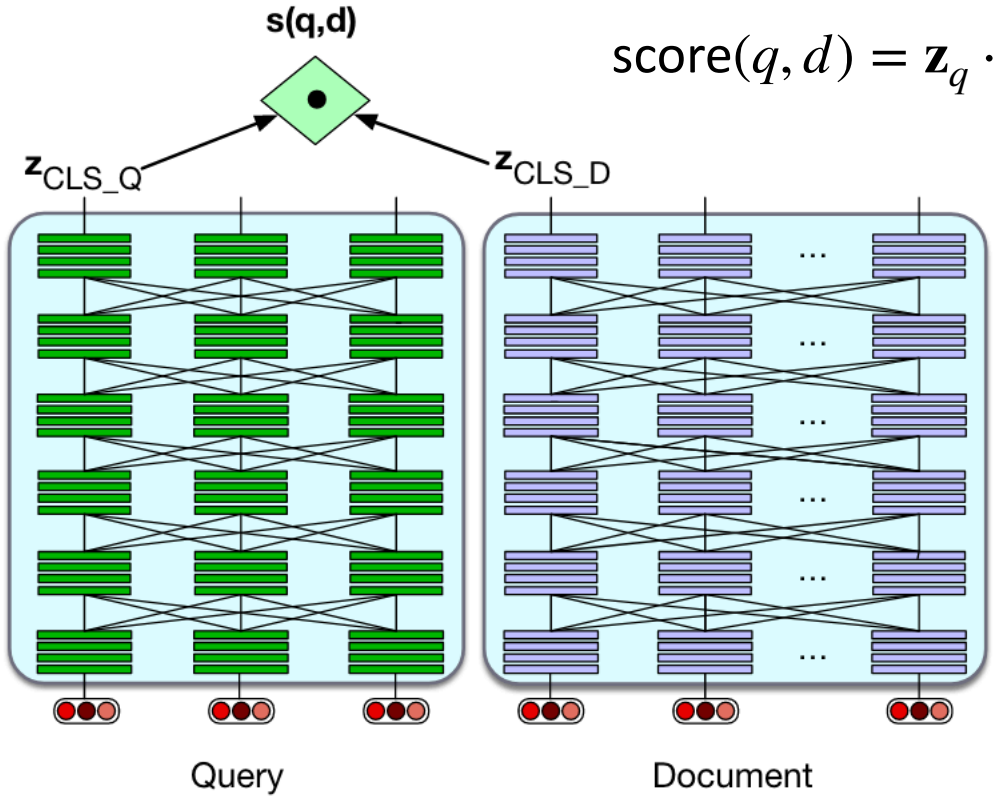
# Information retrieval with pre-trained LMs

- Documents are encoded in advance
- When a new query comes in, we only have to encode that query
- Much more efficient but tends to give worse results
- Question: How to combine the two approaches?
  - **Re-ranking**

$$\mathbf{z}_q = \text{BERT}_{\text{query}}(q)[\text{CLS}]$$

$$\mathbf{z}_d = \text{BERT}_{\text{doc}}(d)[\text{CLS}]$$

$$\text{score}(q, d) = \mathbf{z}_q \cdot \mathbf{z}_d$$



# Efficiency using Reranking: In-between methods

1. Use cheap methods (like BM25) as first pass relevance ranking for each document
2. Then just rerank the top N ranked docs using expensive methods like the full BERT scoring

# Efficiency with Dense retrieval

Unlike with sparse retrieval and inverted index, w**e must rank every document for its similarity to the query!**

**Efficiency for dense retrieval:**

- **nearest neighbor search**: finding the set of dense document vectors that have the highest dot product with a dense query vector.
- Approximate nearest neighbor algorithm **FAISS**(Johnson et al., 2017) on GPUs
  - Approximates the doc vector by a smaller quantized vector

# Retrieval-Augmented Generation (RAG)

**Information retrieval plays a central role in modern LLMs.**

How to answer factual questions like:

- Where is the Louvre Museum located?
- How to get a script l in latex?
- Where does the energy in a nuclear explosion come from?

# Prompting an LLM for information

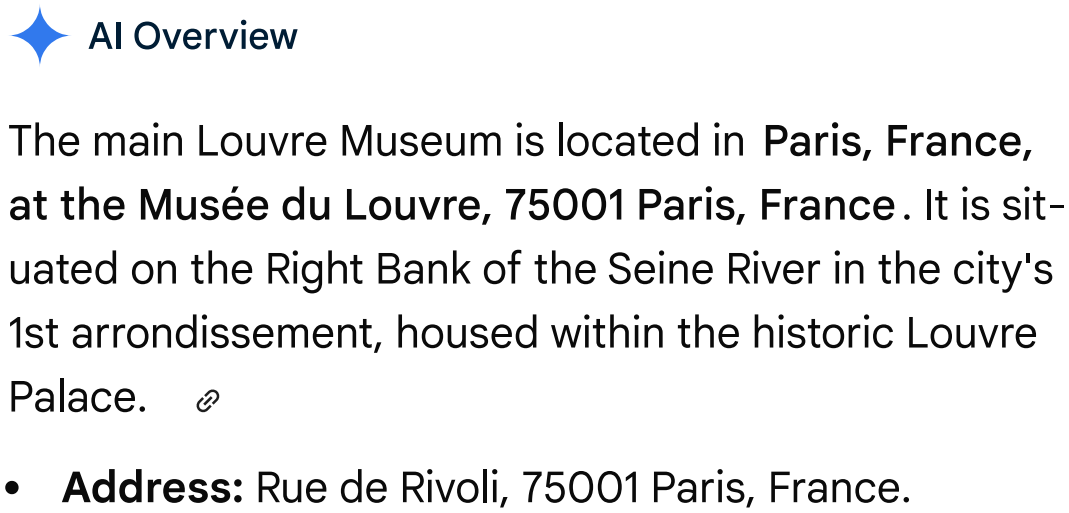
AI Overview

The main Louvre Museum is located in **Paris, France, at the Musée du Louvre, 75001 Paris, France**. It is situated on the Right Bank of the Seine River in the city's 1st arrondissement, housed within the historic Louvre Palace.

- **Address:** Rue de Rivoli, 75001 Paris, France.

# Prompting an LLM for information

LLMs seem to store facts in the connections in their feedforward layers!



# Issue: LLMs Hallucinate

Hallucination: a response that is not faithful to the facts of the world.

In the legal domain LLMs were shown to hallucinate up to 88% of the time!

Dahl et al. (2024)

# Issue: Can’t use Proprietary Data

People need to ask questions about:

- personal email.
- healthcare applications to medical records.
- internal corporate documents
- legal documents discovery

# Issue: Can’t Handle Dynamic Data

LLMs can't answer questions about rapidly changing information

Things that happened last week

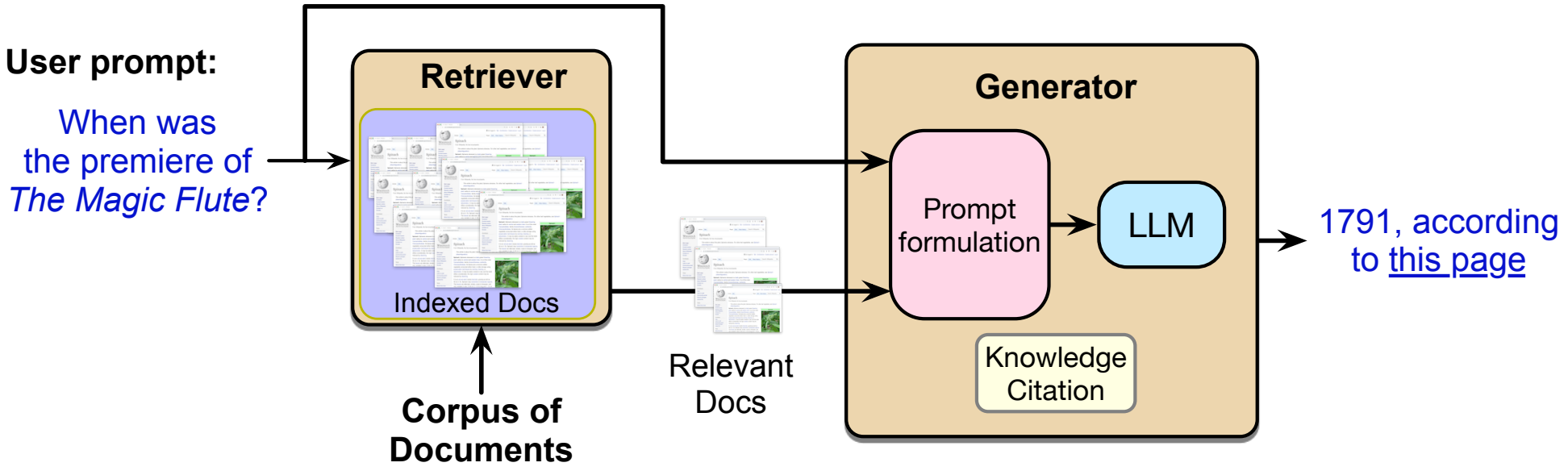
In general, data shifts over time

# Solution: RAG

Retrieval-Augmented Generation

1. Use IR to **retrieve** documents from some collection
2. Then use LLM to **generate** an answer conditioned on the documents

# Retrieval Augmented Generation (RAG)



# Basic RAG

Given a document collection D and a user query q

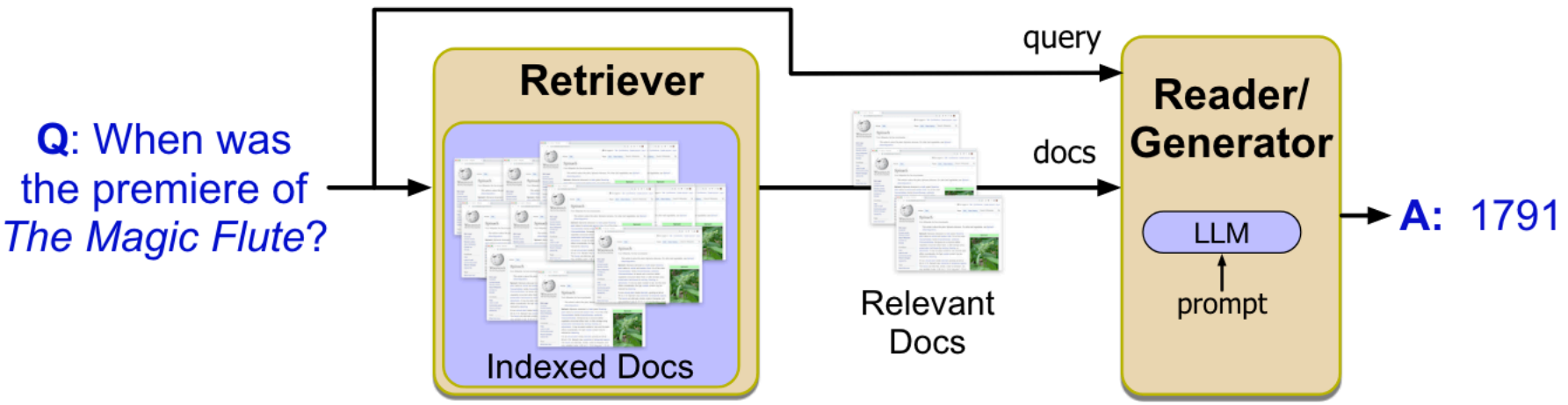
- Call a retriever to return top *k* passages
- Create a prompt that includes *q* and the passages
- Call an LLM with the prompt

**Schematic of a RAG Prompt**

```
retrieved passage 1

retrieved passage 2

...

retrieved passage k

Based on these texts, answer this question:  What year
was the premiere of The Magic Flute?
```

# Information retrieval with pre-trained LMs

- Question Answering (QA) as a retrieval problem
- Find supporting documents for a query
- Extract or generate an answer based on the top-k docs



# Bettering dense retrieval through training

- Consider the document on the right
- There exist many queries for which this document might be relevant
  - When was HEC founded?
  - Can I do a PhD at HEC?
  - What's Viger Square?
  - Etc.
- Ideally, all of this needs to be encoded in the document representation

**HEC Montréal** (French: *Hautes études commerciales de Montréal*; English: *High Commercial Studies of Montreal*) is a bilingual public business school located in Montreal, Quebec, Canada. Founded in 1907, HEC Montréal is the graduate business school of the Université de Montréal and is the first established school of management in Canada.[2][3]

HEC Montréal offers undergraduate, graduate, and post-graduate programs, including Bachelor of Business Administration (BBA), Master of Science in Administration (MSc), Master of Management (MM), Master of Business Administration (MBA), and PhD in Administration, in addition to a joint Executive MBA program with McGill University.

HEC Montréal was founded in 1907 by the Board of Trade of Metropolitan Montreal. Its initial building in Viger Square is now called the Gilles Hocquart Building.[2]

In 1988, a group of HEC students established Jeux du Commerce, where more than 1300 students from 14 universities in Eastern Canada gather annually for academic, social, and sports events.[4][5] A similar competition has been established in Western Canada called JDC West.

As of 2021, the centenary of the HEC Montréal Alumni Association, the school had over 100,000 alumni.[6]

# Bettering dense retrieval through training

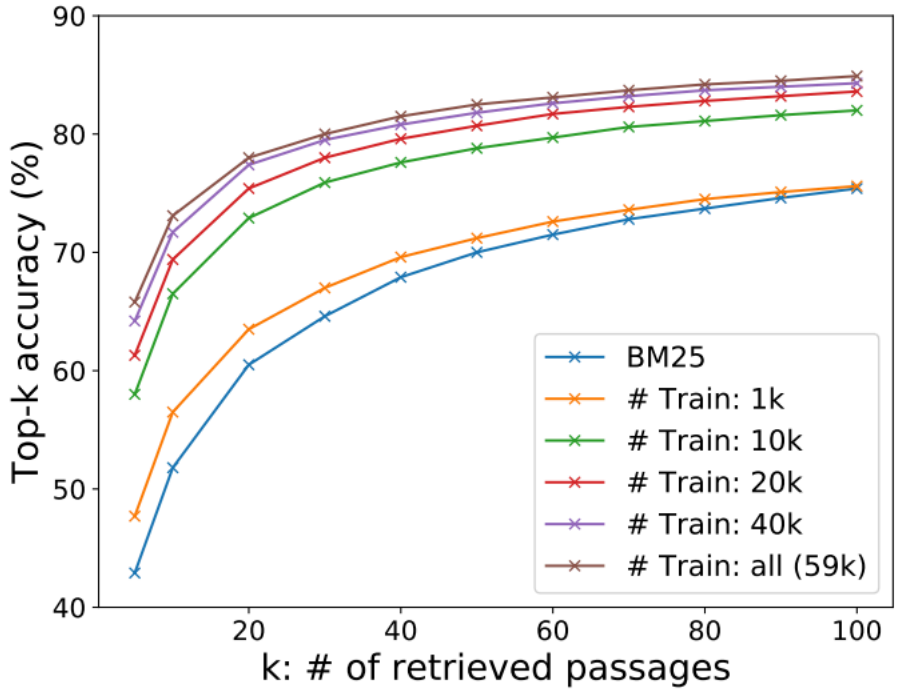
- Use a collection $\{(q_i, d_i^+, d_i^-)\}_{i=1}^n$ of queries paired with positive and negative documents
- Learn an implicit relevance definition based on this collection via **contrastive learning**
- Intuition: move positive documents close to query, push negative documents away from query

$$\mathscr{L}(q_i, d_i^+, \{d_{i,j}^-\}_{j=1}^k) = \frac{\exp(\mathbf{q} \cdot \mathbf{d}_i^+)}{\exp(\mathbf{q_i} \cdot \mathbf{d_i}^+) + \sum_{j=1}^k \exp(\mathbf{q}_i \cdot \mathbf{d}_{i,j}^-)}$$

Karpukhin et al. (2020) - Dense Passage Retrieval for Open-Domain Question Answering

# Dense Passage Retrieval for Open-Domain QA

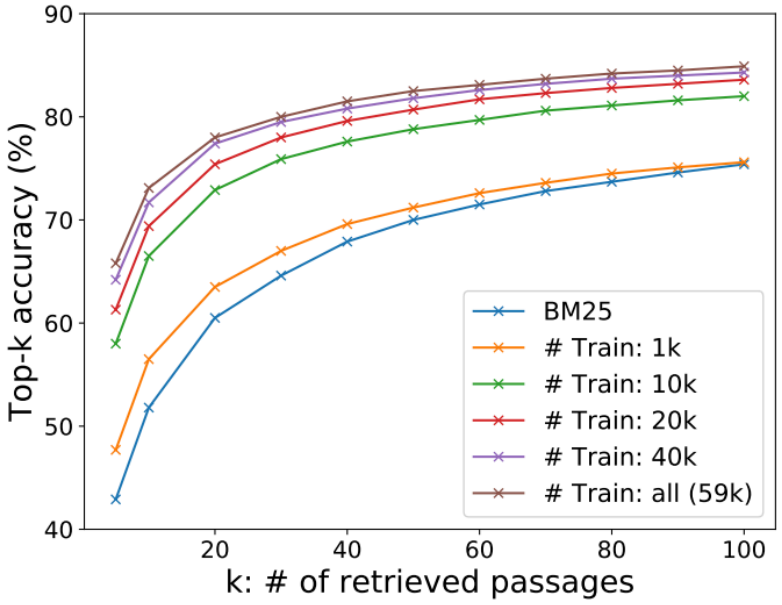
- How does it compare to BM25?

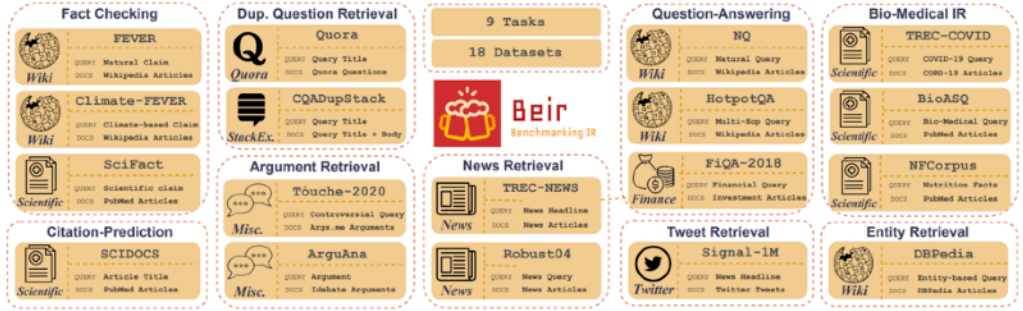


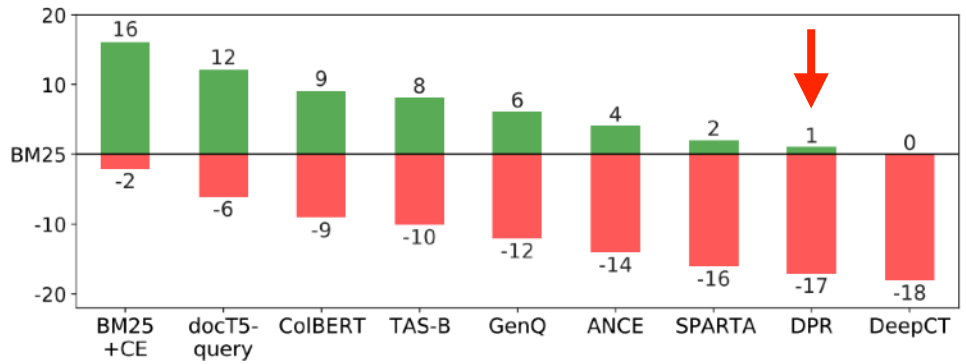

Karpukhin et al. (2020) - Dense Passage Retrieval for Open-Domain Question Answering

# Dense Passage Retrieval for Open-Domain QA

- What happens outside of the training distribution?
- Question: what could be the reason?




Thakur et al. (2021) - BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models

# [15 minute break]

# Case study: An insurance buddy

Your client is an insurance company. Evaluating claims fairly takes a lot of training and accumulated knowledge, but the company faces a lot of employee turnover (insurance isn't always the happiest of jobs!) which means that this knowledge keeps getting lost and training new employees is costly. As a solution to cut down on knowledge transfer time, you propose to create an AI assistant that will help claim agents with an initial assessment. It's role will be to identify (1) the most likely type of claim; (2) the sections in the client's policy that refer to this type of claim. It should then (3) generate an initial assessment of eligibility of the claim and possible resolutions. You have access to the following documents: (i) clients written claims and their full policies (mapped already), (ii) the set of all resolved claims and their resolutions from the past 5 years, (iii) a list of claim categories.

# Case study: An insurance buddy

- Sketch out the procedural structure of your AI Assistant, outlining each step and what NLP technologies will be necessary. If multiple are possible (eg. different LMs could be used) highlight which you would choose and why?
- Identify the limitations of your solutions and where they may introduce errors. What may be your level of responsibility and accountability for these errors?
- Consider possible challenges that your solution may present to data sovereignty, client privacy, and compute costs to the company.